Recursion problems

Problem 1: Find the sum of digits of a number using recursion.

```
#include <stdio.h>

int sum(int n) {
    if (n == 0)
        return 0;
    return (n % 10) + sum(n / 10);
}

int main() {
    int n = 1234;
    printf("Sum of digits: %d\n", sum(n));
    return 0;
}
```

Problem 2: Write a recursive function to reverse a given string.

```
#include <stdio.h>
#include <string.h>

void reverseString(char str[], int i, int j) {
    if (i >= j)
        return;
    char temp = str[i];
    str[i] = str[j];
    str[j] = temp;
    reverseString(str, i + 1, j - 1);
}

int main() {
    char str[] = "hello";
    reverseString(str, 0, 5 - 1);
    printf("Reversed: %s\n", str);
    return 0;
}
```

Problem 3: Compute x^n using recursion.

```
#include <stdio.h>

int power(int x, int n) {
    if (n == 0)
        return 1;
    return x * power(x, n - 1);
}
```

```
int main() {
    int x = 2, n = 5;
    printf("%d^%d = %d\n", x, n, power(x, n));
    return 0;
}
```

Problem 4: Use recursion to find the maximum element in an array of n elements.

```
#include <stdio.h>

int max(int arr[], int n) {
    if (n == 1)
        return arr[0];

    int subMax = max(arr, n - 1);
    if (arr[n - 1] > subMax)
        return arr[n - 1];
    else
        return subMax;
}

int main() {
    int arr[] = {4,5,9,7,1};
    int n = 5;
    int result = max(arr, n);
    printf("This is max number %d", result);
    return 0;
}
```

Problem 5: Write a recursive function to check if a string is palindrome.

```
#include <stdio.h>
#include <string.h>

int isPalindrome( char str[],int i, int j) {

if (i>=j)
return 1;
if (str[i]!=str[j])
return 0 ;
return isPalindrome(str, i+1, j-1) ;
}
int main() {
    char str[] = "madam";
    int n = strlen(str);
    int i = 0, j = n - 1;
```

```
    if (isPalindrome(str, i, j))
        printf("This %s is Palindrome\n", str);
    else
        printf("This %s is not Palindrome\n", str);
    return 0;
}
```

Problem 6: Write a recursive function to return the sum of all elements in an array.

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    if (n == 0)
        return 0;
    return arr[n - 1] + sumArray(arr, n - 1);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = 5;
    printf("Sum of array: %d\n", sumArray(arr, n));
    return 0;
}
```

Problem 7: Implement binary search recursively.

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int key) {
    if (low >= high)
        return -1;
    int mid = (low + high) / 2;
    if (arr[mid] == key)
        return mid;
    else if (key < arr[mid])
        return binarySearch(arr, low, mid - 1, key);
    else
        return binarySearch(arr, mid + 1, high, key);
}

int main() {
    int arr[] = {1, 3, 5, 6, 7, 11};
    int n = 6, key = 7;
    int low=0;
    int high=n-1;
    int result = binarySearch(arr, 0, n - 1, key);
    if (result != -1)
        printf("Found at index %d\n", result);
```

```
    else
        printf("Not found\n");
    return 0;
}
```

Problem 8: A person can climb 1 or 2 steps at a time. Find total ways to reach the top of n steps.

```
#include <stdio.h>

int climbStairs(int n) {
    if (n <= 1)
        return 1;
    return climbStairs(n - 1) + climbStairs(n - 2);
}

int main() {
    int n = 9;
    printf("Total ways to climb %d steps: %d\n", n, climbStairs(n));
    return 0;
}
```

Problem 9: Write a recursive function to find the sum 1 + 2 + … + n.

```
#include <stdio.h>

int sum(int n) {
    if (n == 1)
        return 1;
    return n + sum(n - 1);
}

int main() {
    int n = 5;
    printf("Sum of 1st %d numbers are = %d\n",n, sum(n));
    return 0;
}
```